ÉCOLE SUPÉRIEURE
DE COMMERCE
Sfax

Support de cours
SGBD

Proposé par : Maher EL'ARBI
Auditoire : 2 LIG

Année universitaire 2025-2026

Plan du cours

Chapitre 1 : Introduction au SGBD

1. Qu'est-ce qu'un SGBD
   1.1. Les composants d'un SGBD
   1.2. Les types de SGBD
   1.3. Les fonctions d'un SGBD
2. Le SGBD Oracle
3. SQL

Chapitre 2 : Définition de données

1. Création de schéma d'une base de données
2. Modification du schéma d'une base de données
3. Autres opérations sur les tables

Chapitre 3 : Manipulation des données

1. Insertion de données
2. Mise à jour de données
3. Suppression de données

Chapitre 4 : Interrogation des données

1. Expression des projections
2. Expression des restrictions
3. Expression des jointures
   3.1. Jointure interne
   3.2. Jointures externes
4. Les sous interrogations ou Sous-requêtes
   4.1. Sous-requête ramenant une seule valeur
   4.2. Sous-requête ramenant plusieurs valeurs
      Le prédicat IN
      Le prédicat ALL ou ANY
      Le prédicat EXISTS
   4.3. Sous-interrogation synchronisée
5. Les Opérateurs ensemblistes
6. Les fonctions de calculs
7. Le partitionnement
8. Tri du résultat d'un SELECT
9. Fonctions utiles
10. Les vues

Chapitre 5 : PL/SQL

1. Qu’est-ce que PL SQL
2. Les types des variables
3. Les instructions
4. Gestion des erreurs
5. Procédures et fonctions
6. Les curseurs

Chapitre 6 : Les curseurs explicites

1. Déclaration du curseur
2. Ouverture du curseur
3. Lecture du curseur
4. Fermeture du curseur

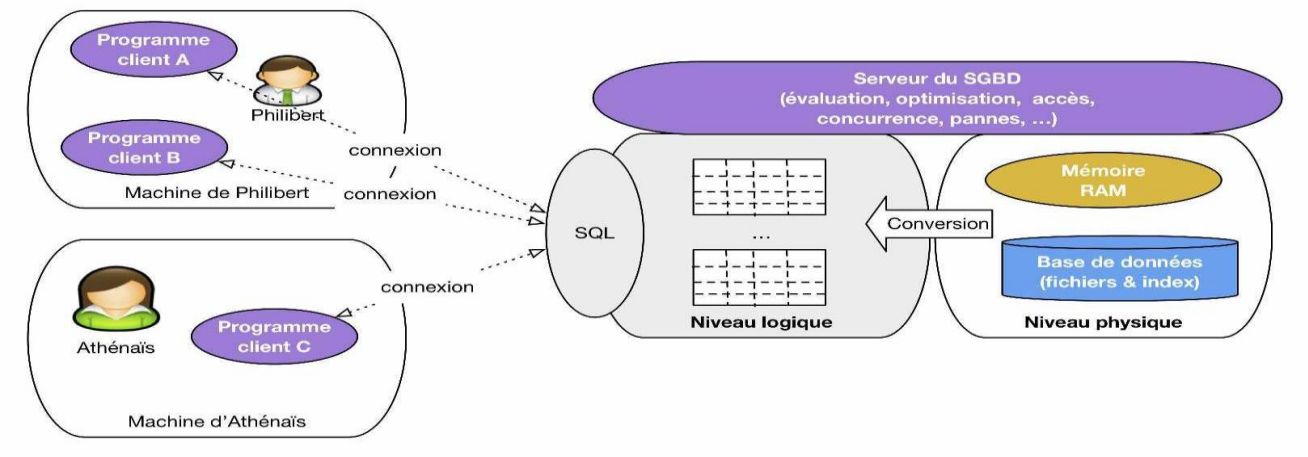
Chapitre 7 : Les déclencheurs

CHAPITRE 1

Introduction aux SGBD

1. Qu‘est-ce qu’un SGBD ?

Une Base de données (DB) : est une collection de données inter reliées, stockées ensemble pour servir une ou plusieurs applications, en parallèle, de façon optimale.
Système de Gestion de Bases de Données (SGBD) ou Data Base Management System (DBMS) : Collection de logiciels permettant de créer, gérer et interroger de manière efficace une BD indépendamment du domaine d‘application.



Le SGBD présente aux applications les données selon un modèle qui fait abstraction de tous les détails techniques de la représentation physique. Dans les SGBD relationnels, ce modèle comprend des tables constituées de lignes.
Le niveau logique est celui de la présentation des données selon ce modèle. La distinction entre niveau logique et niveau physique offre d’immenses avantages qui expliquent en grande partie le succès des systèmes relationnels et la très grande simplification qu’ils ont apportée à la gestion de données informatiques.

1.1. Les composants d'un SGBD

Les SGBD sont des logiciels complexes qui offrent un ensemble complet et cohérent d’outil de gestion de données :

- Un langage de manipulation et d’interrogation (SQL par exemple),
- Un gestionnaire de stockage sur disque,
- Un gestionnaire de concurrence d’accès,
- Des interfaces de programmation et d'administration, .etc.

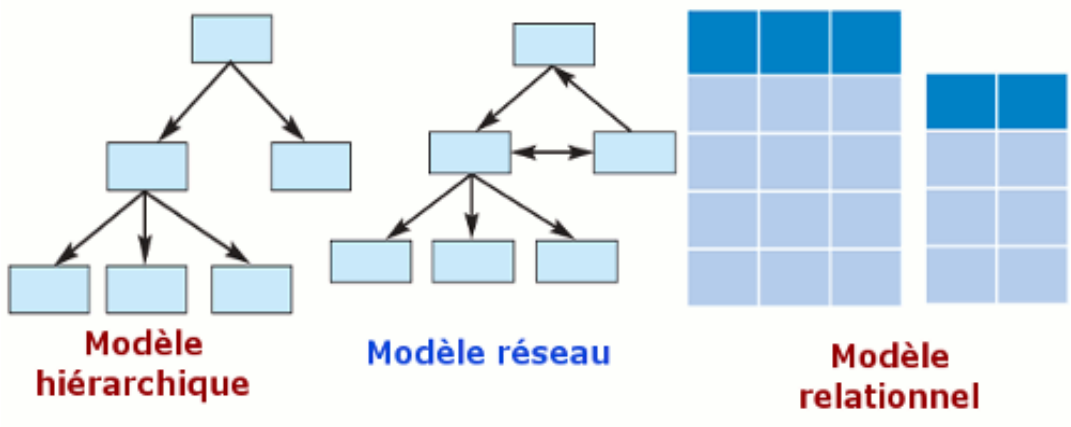
1.2. Les fonctions d’un SGBD

Un SGBD permet de :

- Décrire les données indépendamment des applications (de manière intrinsèque)
- Manipuler les données interroger et mettre à jour les données sans préciser d ‘algorithme d ‘accès
- Contrôler les données intégrité (vérification de contraintes d ‘intégrité) et confidentialité (contrôle des droits Julie, autorisation)
- Partage de BD entre plusieurs utilisateurs en même temps ⇒ contrôle des accès concurrents

- Sécurité reprise après panne, journalisation
- Indépendance physique pouvoir modifier les structures de stockage ou les index sans que cela ait de répercussion au niveau des applications
- Indépendance logique permettre aux différentes applications d 'avoir des vues différentes des mêmes données.

1.4. Les différents types de SGBD

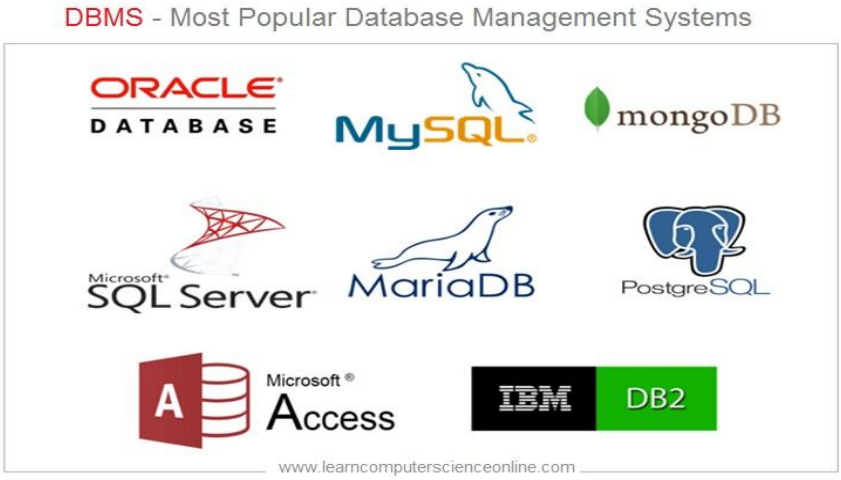


SGBD hiérarchique : les données sont représentées dans la base sous la forme d 'un arbre.
SGBD réseau : les données sont représentées dans la base sous la forme d 'un graphe quelconque.
SGBD relationnel (SBDR) : les données sont représentées dans la base sous la forme de tables.
SGBD relationnel objet (SGBDRO) : les données sont représentées dans la base sous la forme de TAD.
SGBD objet (SGBDOO) : les données sont représentées dans la base sous la forme d'une collection d'objets persistants.
SGBD NoSQL : non relationnelle conçue pour traiter de grands volumes de données non structurées ou semi-structurées.

1.5. Exemples de SGBD

De nombreux SGBD existent sur le marché :

- ACCESS
- MySQL
- PostgreSQL
- ORACLE
- DB2
- Mongo DB



2. Le SGBD ORACLE

Oracle https://www.oracle.com/fr/ est un SGBDR(O) édité en 1979 par la société ORACLE leader mondial des bases de données. Il est écrit en langage C et se décline en plusieurs versions :

- Express Edition (XE),
- Standard Edition One.
- Standard Edition (SE2),
- Enterprise Edition (EE),
- Personal Edition (PE).

2.1. Les fonctionnalités d' ORACLE

Oracle est un SGBD permettant d'assurer :

- La définition et la manipulation des données
- La cohérence des données
- La confidentialité des données
- L'intégrité des données
- La sauvegarde et la restauration des données
- La gestion des accès concurrents

2.2. Architecture fonctionnelle d'ORACLE

L'architecture fonctionnelle d'Oracle repose sur un modèle en couches : les applications se servent d'outils (de développement d'applications, de communication, de génie logiciel, d'administration ou d'aide à la décision) qui utilisent directement SQL ou qui s'appuient sur le PL/SQL, langage procédural d'Oracle qui transforme ses instructions en SQL, les ordres SQL sollicitant finalement le moteur (ou noyau) d'Oracle qui quant à lui accède au dictionnaire des données.

2.3. Les outils d'ORACLE

- Outils d'administration : Oracle manager (SQL*DBA), Network manager, Oracle enterprise manager, Import/export
- Outils de développement : Oracle designer, oracle developer (oracle forms, oracle reports, oracle graphics, procedure builder), sql*plus
- Outils de programmation: Pro*c, pro*cobol, pro*Fortran,Pro*Pascal, Pro*PLI....
- Les outils de communication
- Les outils de génie logiciel

- Les outils d ‘aide à la décision

2.4. Les objets du SGBD ORACLE

Les objets du SGBD Relationnel ORACLE sont les suivants :
- les Tables,
- les Vues,
- les Index,
- les Séquences,
- les Synonymes,
- les Clusters.

3. SQL (Structured Query Language)

Introduit par IBM, évolution du langage SEQUEL, commercialisé tout d'abord par ORACLE. SQL est devenu le langage standard pour décrire et manipuler les Bases de Données Relationnelles.

3.1. Les commandes SQL

- ➔ De définition des données : **CREATE, DROP, ALTER**
- ➔ De manipulation des données : **SELECT, INSERT, UPDATE, DELETE**
- ➔ De contrôle des données : Contrôle des accès concurrents **COMMIT, ROLLBACK**
- ➔ De Contrôle des droits d’accès **GRANT, REVOKE.**

3.2. Utilisation de SQL

SQL peut être utilisé de 2 manières :

- En mode interactif pour apprendre le langage. SQL est un langage pour les développeurs qui n'est pas destiné à un utilisateur final. Les requêtes sont envoyées à partir d'un terminal interactif auquel les résultats sont retournés.

```
Oracle SQL*Plus
File  Edit  Search  Options  Help
Connected to:
Oracle9i Enterprise Edition Release 9.2.0.1.0 - 64bit Production
With the Partitioning option
JServer Release 9.2.0.1.0 - Production

SQL> select * from suppliers
  2  where supplier_id = &supplier_id;
Enter value for supplier_id: 1
old   2: where supplier_id = &supplier_id
new   2: where supplier_id = 1

SUPPLIER_ID SUPPLIER_NAME
----------- ------------------------------------------------
          1 Microsoft

SQL>
```

- En mode intégré dans un L3G hôte (COBOL, ADA, C, FORTRAN …) pour développer des applications.

```
CustomerAdmin.java
Source  History
import injection.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;


/**
 *
 * @author bowsnickiklewe
 */
public class CustomerAdmin {

    public static String getCustomer(String CustID) throws NotFoundException {
        final String driverName = "com.mysql.jdbc.Driver";
        final String dbURL = "jdbc:mysql://localhost/Customers?user=abc&password=abc";
        try {
            Class.forName(driverName);
            Connection conn = DriverManager.getConnection(dbURL);
            Statement stmt = conn.createStatement();
            String sql = "SELECT CName from Customers WHERE CustID = " + CustID;
            System.out.println(sql);
            ResultSet rs = stmt.executeQuery(sql);
            if (rs.next()) {
                return rs.getString(1);
            }
        } catch (Exception ex) { }
            throw new NotFoundException();

    }
```

3.3. Importance du langage SQL

Standard d'accès aux serveurs de données relationnels, norme ISO SQL est le langage commun de nombreux systèmes commercialisés. SQL est l'interface logiciel/logiciel entre les applications et les BDR.
Plusieurs niveaux de normalisation

- **SQL1** : norme de base
- **SQL2** : extension de SQL1 meilleur support des règles du relationnel, types de données plus variés
- **SQL3** : intégration du modèle objet

Les avantages de la normalisation sont :

- Réduction des coûts de formation
- Portabilité des applications
- Pérennité des applications
- Communication facilitée entre systèmes

CHAPITRE 2

Définition des données

1. Création de tables

La commande **CREATE TABLE** crée la définition d'une table.

Syntaxe :

```
CREATE TABLE nom-table
(
        -- définition des colonnes
        -- définition des contraintes niveau table,
) ;
```

On peut créer une table à partir d'une requête SELECTE

Syntaxe:

```
CREATE TABLE nom-table AS requête ;
```

1.1. Définition des colonnes

Syntaxe :

nom-colonne TypeColonne [DEFAULT valeur] [[CONSTRAINT nom-contrainte] TypeContrainte],

Types des colonnes

- **CHAR** (taille) : Chaîne de m longueur fixe de 1 à 255 octets.
- **VARCHAR2**(taille) : Chaîne de taille variable 1...2000 octets
- **DATE** : format par défaut JJ-MOM-AA
- **LONG** : type texte (taille jusqu'à 2Go)
- **NUMBER (n1[, n2])** : n1 = nombre de digits du décimal (de 1 à 38) et n2 = nombre de digits après la virgule.

Types des contraintes

- **NOT NULL :** La colonne doit contenir une valeur (NULL impossible).
- **UNIQUE :** La valeur de la colonne doit être unique.
- **PRIMARY KEY :** La colonne forme la clé primaire.
- **CHECK (**nom-colonne **condition)** : La valeur de la colonne doit vérifier la condition. La condition de la contrainte CHECK peut utiliser les opérateurs =, <>, <, >, <=, >=., BETWEEN valeur1 AND Valeur2, IN ou NOT IN (liste valeurs) et LIKE (_ remplace un caractère et % remplace une suite de caractères). Une condition composée est construite à l'aide des connecteurs AND, OR et NOT.

On peut définir plusieurs contraintes sur la même colonne. Les contraintes sont séparées par un espace.

1.2.Définition des contraintes niveau table

Une contrainte est placée au niveau table, si elle fait référence à plusieurs colonnes.

Syntaxe :

[CONSTRAINT nom-contrainte] Type,

Types des contraintes

- **UNIQUE** (liste des colonnes) **:** Les valeurs des colonnes sont uniques.
- **PRIMARY KEY** (liste des colonnes) **:** La liste des colonnes forme la clé primaire
- **CHECK (**nom-colonne **condition) :** La valeur de la colonne doit vérifier la condition.

Remarque : Une contrainte qui fait référence qu'à une seule colonne de la table (niveau colonne) peut être placée niveau table (sauf la contrainte NULL).

1.3.Définition d'une contraintes clef étrangère

Syntaxe

Niveau colonne

[CONSTRAINT nom-contrainte] REFERENCES table (colonne) [ON DELETE Intégrité référentielle] [ON UPDATE Intégrité référentielle]

Niveau table

[CONSTRAINT nom-contrainte] FOREIGN KEY (colonne) REFERENCES table (colonne) [ON DELETE Intégrité référentielle] [ON UPDATE Intégrité référentielle]

SQL2 permet de spécifier les actions à entreprendre pour le maintien de l'intégrité référentielle, lors d'une suppression ou d'une modification d'un tuple référencé.

- **CASCADE :** cascader les suppressions ou modification.
- **SET NULL :** rendre nul les attributs référençant.
- **RESTRICT :** rejet de la mise à jour. C'est l'option par défaut.

Exemples

Soit le schéma de la base de données suivante :

Client (IdCli, nom, ville)
Produit (IdPro, nom, marque, prix, qtes)
Vente (#IdCli, #IdPro, datev, qtev)

Le nom du produit doit être non nul et unique.
La ville peut être l'une des villes suivantes : Sfax, Souss ou Tunis.

Créer les tables de la base de données.

…….....

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

Créer une table ClientSfax qui contient les clients de la ville de Sfax.

……

……

2. Modification de la définition d'une table

La modification de la définition d'une table est possible à travers la clause ALTER TABLE.

Syntaxe :

```
ALTER TABLE nom-table
        {ADD définition colonne
        |ADD contrainte niveau table
        |MODIFY définition colonne
        |DROP COLUMN nom-colonne
        |RENAME COLUMN old_name TO new_name
        |ENABLE nom-contrainte
        |DISABLE nom-contrainte
        |DROP nom-contrainte};
```

On peut :

- Ajouter une colonne de type NOT NULL uniquement si la table est vide
- Ajouter une contrainte uniquement au niveau table
- Rétrécir une colonne uniquement si elle est vide
- Passer une colonne de NULL à NOT NULL uniquement si la colonne ne contient pas de valeur NULL

- Supprimer une contrainte par son nom

On ne peut pas

- Modifier une contrainte

Exemples

Ajouter la colonne Tel à la table client.

…….....

…….....

Ajouter la colonne email à la table client avec une contrainte qui vérifie que l'email est valide.

…….....

…….....

Ajouter une contrainte qui vérifie si la quantité vendue est comprise entre 1 et 10.

…….....

…….....

Supprimer la colonne email de la table client

…….....

…….....

3. Suppression d'une table

Pour supprimer une table, nous utilisons la commande *DROP TABLE*.

Syntaxe :

DROP TABLE nom-table ;

Exemple

Supprimer la table ClientSfax.

…….....

4. Afficher la définition d'une table

La commande DESC permet de lister les colonnes d'une table.

Syntaxe :

DESC nom-table ;

Exemple

Afficher la définition de la table produit.

…….....

CHAPITRE 3

Manipulation des données

SELECT, **INSERT**, **UPDATE** et **DELETE** sont les 4 commandes de manipulation des données en SQL :

- Recherche : **SELECT**
- Ajout : **INSERT**
- Mise à jour : **UPDATE**
- Suppression : **DELETE**

1. La commande INSERT

La commande INSERT permet d'ajouter de nouvelles lignes à une table.

Syntaxe :

INSERT INTO table [(liste de colonnes)] {VALUES (liste de valeurs) | requête};

- Dans le cas où la liste de colonnes n'est pas spécifiée, tous les attributs de la table cible doivent être fournis dans l'ordre de déclaration.
- Si seulement certaines colonnes sont spécifiées, les autres sont insérées avec la valeur NULL.
- Une insertion à partir d'une requête permet d'insérer plusieurs lignes dans la table cible à partir d'une autre table.

Exemples

Ajouter le client (5, 'Mourad', 'Sfax',12345678) dans la table client.

…….....

…….....

Ajouter le client (8, 'Mehdi') dans la table client.

…….....

…….....

Ajouter dans une table « temp » de même schéma que la table Vente, toutes les ventes qui sont antérieures au 01- Jan-2024.

…….....

…….....

2. La commande UPDATE

La commande UPDATE permet de changer des valeurs d'attributs de lignes existantes

Syntaxe :

```
UPDATE table
SET liste d'affectations
[WHERE qualification] ;
```

L'absence de clause WHERE signifie que les changements doivent être appliqués à toutes les lignes de la table cible.

Exemples

Augmenter de 20% les prix de tous les produits

…….....

…….....

…….....

…….....

Augmenter de 50% les prix des produits achetés par des clients de Sfax.

…….....

…….....

…….....

…….....

3. La commande DELETE

La commande DELETE permet d'enlever des lignes dans une table

Syntaxe:

```
DELETE
FROM table
[WHERE qualification];
```

L'absence de clause WHERE signifie que toutes les lignes de la table cible sont enlevées

Exemples

Supprimer les ventes antérieures au 01-jan-2010

…….....

…….....

…….....

…….....

Supprimer les ventes des clients de Sfax antérieures au 01-mar-1994.

…….....

…….....

…….....

…….....

CHAPITRE 4

Interrogation des données

La commande **SELECT** permet de rechercher des données à partir de plusieurs tables ; le résultat est présenté sous forme d'une table réponse.

La forme générale de SELECT

SELECT [DISTINCT] liste d'attributs, expressions
FROM liste de tables
[WHERE qualification]
[GROUP BY attributs de partitionnement]
[HAVING qualification de groupe]
[ORDER BY liste de colonnes [ASC | DESC]

Le résultat d'un SELECT est construit suivant les étapes :

- FROM : la clause FROM est évaluée de manière à produire une nouvelle table, produit cartésien des tables dont le nom figure après FROM
- WHERE : le résultat de l'étape 1 est réduit par élimination de toutes les lignes qui ne satisfont pas à la clause WHERE
- GROUP BY : le résultat de l'étape 2 est partitionné selon les valeurs des colonnes dont le nom figure dans la clause GROUP BY
- HAVING : les groupes ne satisfaisant pas la condition HAVINGsont éliminés du résultat de l'étape 3
- SELECT : chacun des groupes génère une seule ligne du résultat

1. Expression des projections

Syntaxe :

SELECT *| [ALL | DISTINCT] nom-colonne | expr [label],.....
FROM table Alias_table;

- On peut introduire dans la clause FROM un synonyme à un nom de table en le plaçant immédiatement après le nom de la table.
- Les noms de table ou les synonymes peuvent être utilisés pour préfixer les noms de colonnes dans le SELECT.
- Les préfixes ne sont obligatoires que dans des cas particuliers (pour une auto-jointure) ; leur emploi est cependant conseillé pour la clarté.
- Une étoile (*) permet de lister tous les attributs.
- SQL n'élimine pas les doublons. Pour éliminer les doublons il faut spécifier DISTINCT
- **expr** peut être une constante (entre ‘ ‘) ou une expression obtenue par des opérations arithmétiques (+, -, *, /) sur les colonnes.
- On peut ajouter un label de colonne constitué d’un mot ou une suite de mots entre « et ».

Exemples

Donner tous les renseignements sur les clients.

…….....
…….....
…….....

Donner les noms, marques et prix des produits.

…….....
…….....

Donner les différentes marques de produit.

…….....
…….....

Donner les références des produits et leurs prix majorés de 20%.

…….....
…….....

2. Expression des restrictions

La condition de recherche (qualification) est spécifiée après la clause **WHERE** par un prédicat.

Syntaxe :

SELECT……..
FROM……
***WHERE* condition ;**

Une condition simple peut être =, <>, <, >, <=, >=, LIKE, BETWEEN v1 AND v2, IN et NULL. Une condition composée est construite à l'aide des connecteurs AND, OR et NOT.

Exemples

Donner le détail du produit numéro 5.

…….....
…….....
…….....

Lister les clients dont le nom comporte la lettre A en 2ième position.

…….....
…….....
…….....

Lister les produits dont le prix est compris entre 5000 et 12000.

…….....
…….....
…….....

Lister les produits de marque IBM, Apple ou Dec

…….....
…….....
…….....

Lister les produits dont le prix est inconnu

…….....
…….....
…….....

Lister les produits de marque IBM dont le prix est inférieur à 12000.

…….....
…….....
…….....

3. Expression des jointures

Une jointure a pour but d'afficher des informations issues de plusieurs tables. Quand on précise plusieurs tables dans la clause FROM, on obtient le produit cartésien des tables.

3.1.Jointure interne :

La Jointure interne ou INNER JOIN permet de lier les enregistrements des 2 tables. C'est l'une des jointures les plus communes. La condition de jointure est exprimée après la clause WHERE.

Syntaxe :

```
SELECT….
FROM table1 t1, Table2 t2,,...
WHERE t1.colonne =t2.colonne AND ....…;
```

SQL2 offre la possibilité de spécifier les jointures internes au niveau de la clause FROM selon la syntaxe suivante :

```
FROM table1 t1 [INNER] JOIN table2 t2 ON t1.colonne= t2.colonne
FROM table1 [NATURAL] [INNER] JOIN table2
```

NATURAL signifie jointure naturelle ou égalité des attributs de même nom.

Exemples

Donner les références et les noms des produits vendus.

…….....
…….....
…….....
…….....

Donner les noms des clients de la même ville que Mourad.

…….....
…….....
…….....
…….....

Donner les noms des clients qui ont acheté le produit de nom P1.

…….....
…….....
…….....
…….....

3.2.Jointures externes

La jointure externe OUTER JOIN permet de retenir les lignes d'une table qui n'ont pas de correspondant dans l'autre table, avec des valeurs nulles associées. On distingue jointure externe gauche, droite et complète selon que l'on retient les lignes sans correspondant des 2 tables ou seulement d'une.

Syntaxe :

```
SELECT  …….
FROM table1 t1, Table2 t2,,...
WHERE t1.colonne(+) =t2.colonne2 AND ....…;
```

Le signe (+) suit la colonne de la table qui ne contient pas l'information recherchée,

SQL2 offre la possibilité de spécifier les jointures externes au niveau de la clause FROM selon la syntaxe suivante :

```
FROM table1 t1 {LEFT| RIGHT|FULL} [OUTER] JOIN table2 t2
ON t1.colonne = t2.colonne
FROM table1 NATURAL {LEFT| RIGHT|FULL} [OUTER] JOIN table2
```

Exemples

Lister tous les clients avec le cas échéant leurs achats.

…….....

…….....

…….....

…….....

4. Les sous-interrogations ou Sous-requêtes

Lorsque dans un prédicat un des 2 arguments n'est pas connu, on utilise les sous-interrogations.

Syntaxe :

```
SELECT ...
FROM ...
WHERE non-colonne Op ?
```

Le ? n'étant pas connu, il sera le résultat d'une sous-requête. C'est la sous-requête de niveau le plus bas qui est évaluée en premier, puis la requête de niveau immédiatement supérieur, ...

La sous-requête peut ramener :

- Une valeur
- Plusieurs valeurs
- Plusieurs colonnes.

Un SELECT peut comporter plusieurs sous-interrogations, soit imbriquées, soit au même niveau dans différents prédicats combinés par des AND ou des OR.

4.1. Sous-interrogation ramenant une seule valeur (monoligne)

Avec une sous interrogations monolignes, on utilise un opérateur monoligne (=, !=, >, >=, <,<=). Une sous-interrogation qui ne ramène aucune ligne se termine avec un code d'erreur. Une sous-interrogation ramenant plusieurs lignes provoquera aussi, dans ce cas, une erreur

Syntaxe :

```
SELECT …..
FROM ….
WHERE  nom-colonne {=|<>|<,|<=| > |>=} (SELECT … FROM… WHERE….);
```

Exemples

Donner les noms des clients qui habitent la même ville que le client numéro 5.

…….....

…….....

…….....

…….....

4.2. Sous-interrogation ramenant plusieurs lignes (multilignes)

Une sous-interrogation peut ramener plusieurs lignes à condition que l'opérateur de comparaison admette à sa droite un ensemble de valeurs. Les opérateurs permettant de comparer une valeur à un ensemble de valeurs sont :

- L'opérateur IN
- Les opérateurs obtenus en ajoutant ANY ou ALL à la suite d'un opérateur de comparaison classique (=, !=, >, >=, <, <=)

Le prédicat IN

Il permet de tester si la colonne appartient au résultat d'une sous-requête.

Syntaxe :

SELECT …..
FROM ….
WHERE nom-colonne **[NOT] IN** (SELECT nom-colonne FROM ... WHERE….);

Exemple

Donner les noms des clients qui ont acheté le produit 1.

…….....
…….....
…….....
…….....

Le prédicat ALL ou ANY

Ils permettent de tester si un prédicat de comparaison est vrai pour tous (ALL) ou au moins un (ANY) des résultats d'une sous-requête.

Syntaxe :

SELECT….
FROM …..
WHERE nom-colonne **{=|<>|<,|<=| > |>=}{ALL| ANY**} (SELECT nom-colonne FROM ..WHERE…..);

Exemples

Donner les numéros des clients ayant acheté un produit en quantité supérieure à chacune des quantités de produits achetées par le client 5.

…….....
…….....
…….....
…….....

Donner les numéros des clients ayant acheté un produit en quantité supérieure à au moins l'une des quantités

de produits achetées par le client 5.

…….....

…….....

…….....

…….....

Le prédicat EXISTS

Le prédicat EXISTS (NOT EXISTS) permet de tester si le résultat d'une sous-requête est vide (Faux) ou non (Vrai).

Syntaxe :

```
SELECT….
FROM …..
WHERE [NOT] EXISTS (SELECT … FROM…WHERE…..);
```

Exemples

Donner les noms des produits qui n'ont pas été achetés.

…….....

…….....

…….....

…….....

Donner les noms des produits qui ont été achetés par tous les clients de Sfax.

…….....

…….....

…….....

…….....

…….....

4.4. Sous-interrogation ramenant plusieurs colonnes (multicolonnes)

Il est possible de comparer le résultat d'un SELECT ramenant plusieurs colonnes à une liste de colonnes. La liste de colonnes figurera entre parenthèses à gauche de l'opérateur de comparaison.

Syntaxe :

```
SELECT….
FROM …..
WHERE (liste colonnes) {=| <>} (SELECT liste colonnes FROM…WHERE…..);
```

Exemple

.

…….....
…….....
…….....
…….....

4.5. Sous-interrogation synchronisée

Lorsque la condition de la sous-interrogation fait référence à une colonne de l'interrogation principale, on utilise les sous-interrogations synchronisées.

Exemple

Donner la liste des produits achetés par les clients qui habitent la ville de Sfax.

…….....
…….....
…….....
…….....

5. Les Opérateurs ensemblistes

SQL1 permet d'exprimer l'opération d'union, intersection et différence en connectant des SELECT par des UNION, INTERSECT et MINUS.

Syntaxe :

SELECT ….. FROM …. WHERE
MINUS | INTERSECT | UNION
SELECT … FROM…WHERE….;

Exemples

Donner mes numéros des produits de marque IBM ou ceux achetés par le client numéro 5.

…….....
…….....
…….....
…….....

6. Les fonctions de calculs

SQL fournit des fonctions de calcul opérant sur l'ensemble des valeurs d'une colonne de table.

- COUNT nombre de valeurs
- SUM somme des valeurs

- AVG moyenne des valeurs
- MAX plus grande valeur
- MIN plus petite valeur

On peut faire précéder l'argument du mot clé DISTINCT pour indiquer que les valeurs redondantes doivent être éliminées avant application de la fonction.

Fonction spéciale COUNT (*) compte toutes les lignes dans une table.

Les valeurs nulles ne sont pas prises en compte, sauf pour COUNT(*).

Si l'argument est un ensemble vide, COUNT renvoie la valeur 0, les autres fonctions renvoyant la valeur NULL.

Exemples

Donner le nombre total de clients.

…….....
…….....

Donner le nombre total de clients ayant acheté des produits

…….....
…….....

Donner le nombre total de 'PS1' vendus

…….....
…….....
…….....

Donner les noms des produits moins chers que la moyenne des prix de tous les produits.

…….....
…….....
…….....
…….....

7. Le partitionnement

Le partitionnement permet de regrouper les lignes résultat en fonction des différentes valeurs prises par une colonne spécifiée.

La clause **GROUP BY** permet de partitionner une table en plusieurs groupes. Toutes les lignes d'un même groupe ont la même valeur pour la liste des attributs de partitionnement spécifiés après GROUP BY.

Syntaxe :

SELECT…
FROM…
WHERE….
GROUP BY attributs de partitionnement
[HAVING condition] ;

- La clause **HAVING** permet de spécifier une condition de restriction des groupes. Elle sert à éliminer certains groupes, comme WHERE sert à éliminer des lignes.
- Les fonctions de calcul opèrent sur chaque groupe de valeurs.
- SQL oblige de faire apparaître dans la clause GROUP BY toutes les colonnes qui sont mentionnées dans la clause SELECT.
- Un seul GROUP BY est autorisé par requête.
- Pas de GROUP BY dans une sous-requête.

Exemples

Donner pour chaque référence de produit la quantité totale vendue.

…….....
…….....
…….....
…….....
…….....

Donner la quantité totale achetée par chaque client (0 pour ceux qui n'ont rien acheté)

…….....
…….....
…….....
…….....
…….....

Donner les noms des marques dont le prix moyen des produits est < 5000.

…….....
…….....
…….....
…….....
…….....

Donner les références des produits achetés en qte > 10 par plus de 50 clients.

…….....
…….....
…….....
…….....

…….....

Donner mes numéros, les prix, les marques et la quantité maximum vendue de tous les produits IBM, Apple ou Dec dont la quantité totale vendue est supérieure à 500 et dont les quantités vendues sont > 10.

…….....

…….....

…….....

…….....

…….....

…….....

8. Tri du résultat d'un SELECT

La clause ORDER BY permet de spécifier les colonnes définissant les critères de tri

Syntaxe :

SELECT…
FROM…
WHERE…
GROUP BY…
HAVING….
ORDER BY colonne1 [ASC | DESC], colonne2 [ASC |DESC] ;

Le tri se fera d'abord selon la première colonne spécifiée, puis selon la deuxième colonne etc…L'ordre de tri est précisé par ASC (croissant) ou DESC (décroissant) ; par défaut ASC

Exemples

Lister les produits en les triant par marques et à l'intérieur d'une marque par prix décroissants.

…….....

…….....

…….....

…….....

9. Fonctions utiles

Fonctions numériques

- ABS(n) : valeur absolue de n
- SIGN(n) : signe de n (-1 ou 0 ou +1)
- CEIL(n) : plus petit entier >= n
- FLOOR(n) : plus grand entier <= n
- MOD (m, n) : reste de la division de m par n
- POWER (m, n) : m élevé à la puissance n
- SQRT(n) : racine carrée de n (message d ‘erreur si n < 0)
- ROUND (n, [m]) : arrondi de n à 10-m
- TRUNC(n,[m]) : n tronqué à 10-m

Fonctions caractères

- LENGTH (chaîne) : longueur de la chaîne
- UPPER (chaîne) : toutes les lettres de la chaîne en majuscules
- LOWER (chaîne) : toutes les lettres de la chaîne en minuscules
- INITCAP (chaîne) : première lettre de chaque mot de la chaîne en majuscules, les autres en minuscules)
- LPAD (chaîne, lg, [chaîne]) : compléter à gauche par une chaîne de caractères sur une longueur donnée.
- LTRIM (chaîne [, caractères]) : suppression à gauche de caractères dans la chaîne.
- SUBSTR (chaîne, position [, longueur]) : extraction d ‘une chaîne à partir d ‘une position donnée et sur une longueur donnée
- INSTR (chaîne, sous_chaîne, [, position [, n]]) : recherche de la position de la n ième occurrences d ‘une chaîne de caractères dans une autre chaîne de caractères à partir d ‘une position données.

Fonctions date

- LAST_DAY (date) : dernier jour du mois d ‘une date donnée
- NEXT_DAY (date, jour) : date du prochain jour à partir d ‘une date donnée.
- ADD_MONTHS (date, n) : ajoute n mois à une date donnée.
- MONTHS_BETWEEN (date1, date2) : nombre de mois entre 2 dates.

Fonctions de conversion de types

- TO_NUMBER (chaîne) : conversion d ‘une chaîne de caractères en nombre
- TO_CHAR (val [, ‘format ‘]) : conversion d ‘une expression (date ou numérique) en chaîne de caractères selon un format de présentation.
- TO_DATE (chaîne [, ‘format ‘]) : conversion d ‘une chaîne en date selon un format.

La fonction DECODE

Syntaxe

```
DECODE (expression, valeur1, result1, [, valeur2, result2] ... [, défaut]
```

Exemple

```
SELECT nom, DECODE (tel, null,  ‘Pas de tél ‘,  ‘Tél : ‘|| tel) "Info Tél "
FROM Client;
```

La fonction NVL

Syntaxe

```
NVL (expr, valeur)
```

Exemple

```
SELECT nom, NVL (prix, 0)
FROM Produit;
```

10. Les vues

Une vue est une table logique qui permet l'accès aux données d'une ou plusieurs tables de bases de façon transparente. Une vue ne contient aucune ligne. Les données sont stockées dans les tables.

10.1. Types de vues dans Oracle

Vues standards :

- Ce sont les vues les plus courantes.
- Elles ne stockent pas de données physiquement.

Vues matérialisées :

- Ces vues stockent physiquement les résultats de la requête.
- Elles peuvent être rafraîchies manuellement ou automatiquement à intervalles réguliers.
- Utiles pour améliorer les performances des requêtes complexes.

10.2. Création d'une vue

La syntaxe de base pour créer une vue est la suivante :

Syntaxe

CREATE VIEW nom_de_la_vue
AS SELECT colonne1, colonne2, ...
FROM nom_de_la_table
WHERE condition ;

10.3. Opération sur les vues

Une vue peut être interroger comme une table normale avec un ordre SELECT.

Certaines vues peuvent être mises à jour (INSERT, UPDATE, DELETE) si elles respectent certaines conditions :

- La vue doit être basée sur une seule table.
- La vue ne doit pas contenir de fonctions d'agrégation, de `DISTINCT`, de `GROUP BY`, etc.
- La vue doit inclure toutes les colonnes `NOT NULL` de la table sous-jacente.

Pour modifier la définition d'une vue existante, utilisez CREATE OR REPLACE VIEW nom_vue_existante…AS requête ;

Pour supprimer une vue, utilisez la commande DROP VIEW nom_vue ;

10.4. Avantages des vues

- Sécurité : Restreindre l'accès à certaines données sensibles.
- Simplicité : Masquer la complexité des requêtes SQL.
- Réutilisabilité : Une vue peut être utilisée dans plusieurs requêtes.

- Abstraction : Fournir une interface cohérente même si la structure des tables sous-jacentes change.

10.5. Limites des vues

- Performances : Les vues standards exécutent la requête sous-jacente à chaque accès, ce qui peut être lent pour des requêtes complexes.
- Mise à jour limitée : Toutes les vues ne sont pas modifiables.
- Dépendance : Les vues dépendent des tables sous-jacentes. Si une table est supprimée, la vue devient invalide.

Exemples

Créer une vue Commande qui contient tous les articles commandés par le client numéro 5 entre le 01/01/2025 et 31/01/2025.

…….....
…….....
…….....
…….....

Afficher le montant total des achats du client numéro 5 entre le 01/01/2025 et 31/01/2025.

…….....
…….....
…….....
…….....

CHAPITRE 4

PL/SQL

1. Qu'est-ce que PL/SQL ?

L'utilisateur saisit une requête (en SQL) et Oracle fourni une réponse. Cette façon de travailler ne fonctionne pas dans un environnement de production, car tous les utilisateurs ne connaissent pas ou n'utilisent pas SQL, et il y a souvent des erreurs. Pour surmonter ces limitations, Oracle intègre un gestionnaire PL / SQL au serveur de base de données et à certains de ses outils (formulaires, rapports, graphiques, etc.). Ce langage intègre toutes les caractéristiques des langages de troisième génération : gestion des variables, structure modulaire (procédures et fonctions), structures de contrôle (Boucles et autres structures), contrôle des exceptions et intégration totale dans l'environnement Oracle.

2. Structure d'un programme PL/SQL

Un programme ou une procédure PL/SQL est un ensemble de un ou plusieurs blocs.

DECLARE
    Déclaration
BEGIN
    corps-du-bloc
EXCEPTION
    traitement-des-erreurs
END;
/ ← A ajouté obligatoirement dans l'exécution d'un script

Chaque bloc comporte trois sections :

- Section déclaration (facultative): Contient la description des structures et des variables utilisées dans le bloc. Commence par le mot clé DECLARE.
- Section corps du bloc (Obligatoire): Contient les instructions du programme. Introduite par le mot clé BEGIN. Se termine par le mot clé END.
- Section traitement des erreurs (Facultative): Introduite par le mot clé EXCEPTION.

Chaque instruction se termine par ;
Les commentaires sont possibles : sur une ligne (–) et Sut plusieurs lignes (/*….. */)
Il est possible d'imbriquer des blocs.

3. Les variables

Variables scalaires

Types SQL: CHAR, NUMBER, DATE, VARCHAR2.
Types PL/SQL: BOOLEAN, SMALLINT, BINARY_INTEGER, DECIMAL, FLOAT,INTEGER, REAL, ROWID.

Les variables hôtes sont préfixées par « : ».

Déclaration des variables scalaires :

nom-variable Type;
nom-variable type := valeur;
nom-variable nom-variable1%TYPE;
nom-variable nom-table.nom-attribut%TYPE;

Déclaration des constantes :

nom-variable nom-type DEFAULT valeur;
nom-variable CONSTANT nom-type := valeur;

Déclaration des variables enregistrement

Soit par référence à une structure de table ou de curseur en utilisant ROWTYPE :

nom-variable nom-table%ROWTYPE;

Soit par énumération des rubriques qui la composent. Cela se fait en deux étapes :

- Déclaration du type enregistrement : TYPE nom-du-type-record IS RECORD (nom-attribut1 type-attribut1, nom-attribut2 type-attribut2, ..);
- Déclaration de la variable de type enregistrement : nom-variable nom-du-type-record;

Déclaration des variables Tableau

Structure composée d'éléments d'un même type scalaire.
L'accès à un élément du tableau s'effectue grâce à un indice, ou clé primaire. Cet index est déclaré de type BINARY_INTEGER (valeur entière signée)

Déclaration du type des éléments du tableau :

TYPE nom-du-type-table IS TABLE² OF type-argument
INDEX BY BINARY_INTEGER;

Déclaration de la variable de type tableau : nom-variable nom-du-type-table;

Visibilité des variables

Une variable est utilisable dans le bloc où elle a été définie ainsi que dans les blocs imbriqués dans le bloc de définition, sauf si elle est redéfinie dans un bloc interne.

Conversion de type

Explicite avec TO_CHAR, TO_DATE, TO_NUMBER, RAWTOHEX, HEXTORAW
Implicite, par conversion automatique.

4. Les instructions

Affectation

Opérateur d'affectation :=
Option INTO dans un ordre SELECT

Structures alternatives

```
IF condition THEN
        instructions;
ELSEIF condition THEN
        instructions;
ELSE
        instructions
END IF;
```

```
CASE expression
WHEN expr1 THEN instructions1;
WHEN expr2 THEN instructions2;
…
ELSE instructionsN;
END CASE;
Expression de type simple
```

Structures répétitives

```
LOOP
        instructions;
        EXIT WHEN Condition;
END LOOP;
```

```
FOR variable-indice IN
[REVERSE] val-début .. val-fin
LOOP
        instructions;
END LOOP;
```

```
WHILE condition
LOOP
        instructions;
END LOOP;
```

5. Gestion des erreurs

Deux types d'erreurs :

- Erreur détectée par le SGBD
- Erreur générée par l'utilisateur.

Chaque catégorie peut être prise en compte dans la section EXCEPTION ou par l'environnement appelant.

5.1. Traitement des erreurs Système

Syntaxe

```
BEGIN
...
EXCEPTION
WHEN nom-erreur1 THEN traitement-erreur1 ;
...
WHEN nom-erreurn THEN traitement-erreurn ;
WHEN OTHERS THEN traitement-autres-erreur ;
END ;
```

On peut utiliser les connecteurs AND, OR dans le cas où le même traitement concerne plusieurs erreurs.

Exemple d'erreurs

- NO_DATA_FOUND
- TOO_MANY_ROWS
- ZERO_DIVIDE

5.2. Erreurs utilisateur

Syntaxe

```
DECLARE
nom-anomalie EXCEPTION;
BEGIN
...
IF ... THEN RAISE nom-anomalie;
…
 EXCEPTION
WHEN nom-anomalie THEN traitement;
END;
```

On peut utiliser la procédure standard RAISE_APPLICATION_ERROR(numéro, texte); Le numéro doit être compris entre -20000 et - 20999.

6. Procédures et fonctions

Une procédure ou fonction peut être déclarée :

- Soit dans la base de données : procédure/fonction stockée
- Soit dans la partie DECLARE d'un bloc PL SQL : procédure/fonction locale

6.1. Déclaration de procédure

Procédure stockée

```
CREATE [OR REPLACE] PROCEDURE nom-procédure [(argument [mode] type, ...)] AS
        Déclaration des variables ;
        BEGIN
        bloc-procedure;
        END nom-procédure;
```

- Argument : nom d'un paramètre formel
- Mode : définit si le paramètre formel est en entrée (IN), en sortie (OUT) ou en entrée- sortie (IN OUT). Par défaut : IN
- Type : le type du paramètre formel
- bloc-procédure : le corps PL/SQL de la procédure

Procédure locale

```
DECLARE
PROCEDURE nom-procédure [(argument [mode] type, ...)] IS
Déclaration variables
BEGIN
Bloc-procedure;
END nom-procédure;
BEGIN
…..
END;
```

6.2. Déclaration de fonction

Fonction stockée

```
CREATE [OR REPLACE] FUNCTION nom-fonction [(argument [IN] type, ...)] RETURN type-retour AS
    Déclaration variables
    BEGIN
    bloc-fonction;
    RETURN nom-variable;
    END nom-fonction;
```

Fonction locale

```
DECLARE
FUNCTION nom-fonction [(argument [IN] type, ...)] RETURN type-retour IS
Déclaration variables
BEGIN
bloc-fonction;
RETURN nom-variable;
END nom-fonction;
BEGIN
…..
END;
```

6.3. Appel d'une procédure/fonction stockée

Dans un bloc PL/SQL :

```
nom-procédure(liste-paramètres-effectifs);
nom-variable := nom-fonction(liste-paramètres- effectifs);
```

Dans un script SQL Developer

```
EXECUTE nom-procédure (liste-paramètres- effectifs);
EXECUTE nom-variable := nom-fonction (liste- paramètres-effectifs);
```

7. Les Curseurs

7.1. C'est quoi un curseur ?

Un curseur est une zone de mémoire privée, temporairement allouée dans la zone globale de la session utilisateur (UGA), qui est utilisée pour traiter les instructions SQL. La mémoire privée stocke l'ensemble des résultats récupérés lors de l'exécution du SQL et les attributs du curseur.
Les curseurs peuvent être classés en curseurs implicites et explicites.

7.2. Curseur implicite

Le curseur implicite est un curseur de session qui est déclaré et géré implicitement par PL/SQL.
Le serveur Oracle ouvre un curseur implicite chaque fois qu'une instruction est exécutée. Cette instruction peut être :

- SELECT, qui doit alors ramener exactement une seule ligne.
- INSERT, DELETE ou UPDATE sans aucune contrainte sur le nombre de n-uplets affectés.

7.3. Attributs d'un curseur implicite

Le programmeur ne peut pas contrôler un curseur implicite, mais il peut obtenir des informations à partir de ses attributs qui peuvent être employés dans une section de traitement ou d'exception. Les principaux attributs sont les suivants :

- SQL%FOUND : Vrai si au moins une ligne est traitée par l'ordre SQL.
- SQL%NOTFOUND : Vrai si aucune ligne est traitée par l'ordre SQL
- SQL%ROWCOUNT : Nombre de lignes traitées par l'ordre SQL.

Exemple

Écrire un programme PL SQL qui permet d'afficher la fiche client de la manière suivante :

Fiche Client
Code : 5
Nom : Mourad
Ville : Sfax
Téléphone : 12345678

Dans le cas où la ville n'est pas renseignée, afficher aucune ville.
Dans le cas où le téléphone n'est pas renseigné, afficher pas de téléphone.
Si le client n'est pas trouvé, afficher le client n'existe pas.

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Écrire une fonction PL SQL (nbAchat) qui permet de retrouver le nombre d'achats effectué par un client donné. Si le client n'existe pas retourner -1. Si le client a effectué nb achats alors retourner nb.

……

……

……

……

……

……

……

……

……

……

Écrire un programme PLSQL qui appel la fonction nbAchats et affiche les messages suivants :

- ✓ Si nbAchats retourne -1 alors afficher client non trouvé.
- ✓ Si nbAchats retourne 0 alors afficher aucun achat effectué par le client.
- ✓ Sinon afficher le client a effectué nb achats.

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

CHAPITRE 5

Les curseurs explicites

1. Qu'est-ce qu'un curseur explicite ?

Un curseur explicite est un curseur de session que le programmeur construit et gère. Il est associé à une requête qui renvoie plusieurs lignes.

2. Pourquoi utiliser les curseurs explicites ?

L'avantage de l'utilisation du curseur explicite par rapport au curseur implicite est que le curseur explicite donne au programmeur un contrôle plus « programmatique ».
En outre, les curseurs implicites SELECT...INTO (ramène une seule ligne) sont moins efficaces que les curseurs explicites car ces derniers ne permettent pas de traiter un ensemble de lignes en même temps.

3. Le processus de travail avec un curseur explicite

Le processus de travail avec un curseur explicite se compose des étapes suivantes :

1. Déclaration du curseur
2. Ouverture du curseur
3. Traitement des lignes du résultat
4. Fermeture du curseur

3.1. Déclaration du curseur

L'association d'un nom de curseur à une requête SELECT se fait dans la section DECLARE d'un bloc PL/SQL de la manière suivante :

```
CURSOR nom-curseur IS requête;
```

Un curseur peut être paramétré : CURSOR nom-curseur (nom-1 type-1 [:= val-défaut], ...) IS requête;

3.2. Ouverture d'un curseur

Alloue un espace mémoire au curseur et positionne les éventuels verrous.

```
OPEN nom-curseur; ou OPEN nom-curseur (liste-par-effectifs);
```

Pour les paramètres, association par position ou par nom sous la forme paramètre-formel => paramètre-réel

3.3. Traitement des lignes

Le parcours du curseur se fait selon la syntaxe suivante :

```
LOOP
FETCH nom-curseur INTO destination;
EXIT WHEN nom-curseur%NOTFOUND;
–Traitement de la ligne courante
END LOOP;
```

Le champ destination peut être soit :

- Une liste de variables de même nombre et type que les colonnes présentent dans l'ordre SELECT.
- Une variable de même structure que le curseur.
- Une variable de type enregistrement ayant la même structure que le curseur.

3.4. Statut d'un curseur

- nom-curseur%FOUND : Vrai si au moins une ligne est renvoyée par le dernier FETCH.
- nom-curseur%%NOTFOUND : Vrai si aucune ligne est renvoyée par le dernier FETCH
- nom-curseur%ISOPEN : Vrai si curseur ouvert
- nom-curseur%ROWCOUNT : le nombre de lignes extraites depuis l'ouverture d'un curseur.

3.5. Fermeture du curseur

```
CLOSE nom-curseur;
```

4. La Boucle FOR

La boucle FOR déclare les variables, ouvre, parcourt et ferme le curseur.

```
FOR variable IN nom-curseur
LOOP
Traitement de la ligne courante
END LOOP;
```

Exemple

Écrire un programme PL SQL qui permet d'afficher les achats du client numéro 5 de la manière suivante :

Achats Client

Code : 5

Nom : Mourad

Date achat	Produit	Quantité	Prix unitaire	Total
01-jan-2023	Flash Disk 16 GB	10	15	150
01-jan-2023	Flash Disk 64 GB	10	30	300

Total quantité : 20

Total achat : 450

…….....

CHAPITRE 6

Les déclencheurs (triggers)

1. Qu'est-ce qu'un déclencheur

Un trigger (déclencheur) est un programme qui se déclenche automatiquement à la suite d'un événement. C'est donc un programme que l'on n'appelle pas explicitement, à la différence d'une procédure stockée. Les triggers sont déclenchés par des instructions LMD qui modifient l'instance d'une table : INSERT, DELETE ou UPDATE.

2. Pourquoi utiliser les déclencheurs

Les triggers peuvent servir à :

- Vérifier des contraintes que l'on ne peut pas définir de façon déclarative
- Gérer de la redondance d'information, après.une dénormalisation du schéma.
- Collecter des informations sur les mises-à-jour de la base.

3. Types de déclencheurs

ORACLE propose deux types de triggers :

- Les triggers de table (STATEMENT) sont déclenchés une seule fois.
- Les triggers de ligne (ROW). Se déclenchent individuellement pour chaque ligne de la table affectée par le trigger.

Si l'option FOR EACH ROW est spécifiée, c'est un trigger ligne, sinon c'est un trigger de table.

4. Déclencheur de table

Syntaxe

```
CREATE [OR REPLACE] TRIGGER nom-déclencheur
AFTER| BEFORE INSERT [OR UPDATE | OR DELETE]
ON nom-table
bloc-PL/SQL;
```

Pour l'événement UPDATE, on peut spécifier les attributs concernés en mettant UPDATE OF nom-attribut1, ...

Un déclencheur de table avec l'option BEFORE peut servir à soumettre l'exécution d'un traitement de mise à jour d'une table à certaines conditions, avec émission d'un message d'erreur si les conditions ne sont pas vérifiées.

Un déclencheur de table avec l'option AFTER peut servir à faire des validations a posteriori afin de vérifier que les modifications se sont bien déroulées. Il peut aussi permettre de propager des modifications dans plusieurs tables.

5. Déclencheur de ligne

Syntaxe

```
CREATE [OR REPLACE] TRIGGER nom-déclencheur
AFTER| BEFORE INSERT [OR UPDATE | OR DELETE]
ON nom-table FOR EACH ROW
[WHEN condition]
bloc-PL/SQL;
```

On peut faire référence, dans la condition WHEN ou dans le bloc PL/SQL associé au déclencheur à la valeur d'un attribut avant modification en préfixant le nom de l'attribut par :OLD, et/ou à la valeur après modification en préfixant le nom de l'attribut par :NEW

La valeur prise en compte dépend de l'ordre SQL :

Ordre SQL	**:OLD**	**:NEW**
INSERT	NULL	Valeur créée
DELETE	Valeur avant suppression	NULL
UPDATE	Valeur avant modification	Valeur après

La clause WHEN ne peut pas contenir de requête SQL

Un déclencheur par ligne avec l'option BEFORE peut servir à effectuer des traitements d'initialisation avant l'exécution des modifications sur la table.

Un déclencheur par ligne avec l'option AFTER permet de propager les modifications ou de gérer l'historique.

6. Operations sur les triggers

Suppression d'un déclencheur : DROP TRIGGER nom-déclencheur;
Activation/Désactivation d'un déclencheur :

```
ALTER TRIGGER nom-déclencheur DISABLE;
ALTER TABLE nom-table DISABLE ALL TRIGGERS;
ALTER TRIGGER nom-déclencheur ENABLE;
ALTER TABLE nom-table ENABLE ALL TRIGGERS;
```

7. Les prédicats conditionnels INSERTING, DELETING et UPDATING

Quand un trigger comporte plusieurs instructions de déclenchement (par exemple INSERT OR DELETE OR UPDATE), on peut utiliser des prédicats conditionnels (INSERTING, DELETING et UPDATING) pour exécuter des blocs de code spécifiques pour chaque instruction de déclenchement.

```
CREATE TRIGGER ...
BEFORE INSERT OR UPDATE ON ….
.......
BEGIN
......
IF INSERTING THEN ..... END IF;
IF UPDATING THEN ..... END IF;
.....
```

END;

8. Ordre d'exécution des déclencheurs

Il est possible d'associer un et un seul déclencheur de chaque type à chaque table. Lorsque plusieurs déclencheurs sont associés à la même table, l'ordre d'exécution est le suivant :

1) Déclencheur de table BEFORE
2) Déclencheur de ligne BEFORE
3) Déclencheur de ligne AFTER
4) Déclencheur de table AFTER

Exemple

Écrire un déclencheur qui permet de mettre à jour la quantité en stock d'un produit vendu.

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……

……